

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2024 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Model 2024	2
SUBIECTUL I (30 de puncte)	2
1. Date de tip real (Floating Point)	2
2. Tehnologia Informației (TIC): Tabele în procesoare de text	2
SUBIECTUL al II-lea (30 de puncte)	2
1. Programare: n-ouroboros	2
2. Algoritm eficient: Sumă subset 2023	5
II. Rezolvare Titularizare Varianta 3 (17 iulie 2024)	6
SUBIECTUL I (30 de puncte)	6
1. Structura de date: Arbori	6
2. Ergonomia postului de lucru	8
SUBIECTUL al II-lea (30 de puncte)	8
1. Programare: Lanț muntos (Creastă)	8
2. Algoritm eficient: Tije discuri	10
SUBIECTUL al III-lea (30 de puncte)	11
1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)	11
2. Evaluare: Itemi obiectivi (Alegere multiplă)	12

I. Rezolvare Titularizare Model 2024

SUBIECTUL I (30 de puncte)

1. Date de tip real (Floating Point)

- **Tip real in C++:** double (8 octeți / 64 biți).
- **Tip real in Pascal:** double sau real (8 octeți în Pascal modern).
- **Patru exemple de declarare cu inițializări (C++ / Pascal):**
 1. Zecimal standard: double a = 3.14; / a: double = 3.14;
 2. Partea fracționară omisă: double b = 5.; / b: double = 5.0;
 3. Partea întregă omisă: double c = .75; / c: double = 0.75;
 4. Reprezentare exponențială (științifică): double d = 1.2e-3; / d: double = 1.2e-3;
- **Exemple de operatori:** adunarea (+), împărțirea reală (/).
- **Funcții predefinite:** radical (sqrt), valoare absolută (abs), rotunjire (round).
- **Problemă (Aria unui cerc):**
 - **C++:**

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double r;
    cin >> r;
    double aria = M_PI * r * r;
    cout << aria << endl;
    return 0;
}
```

- **Pascal:**

```
program AriaCerc;
var r, aria: double;
begin
    read(r);
    aria := pi * r * r;
    writeln(aria);
end.
```

2. Tehnologia Informației (TIC): Tabele în procesoare de text

- **Noțiuni:** Celulă, linie, coloană. Tabelul se poate insera în corpul documentului sau în antet/subsol.
- **Inserare:** Din meniul Insert (Tabel $M \times N$) sau prin desenarea manuală a tabelului.
- **Personalizare:** Îmbinare celule (Merge), divizare (Split), culori de fundal (Shading), borduri (Borders), lățime coloană (Width), înălțime rând (Height), aliniere text în celulă.

SUBIECTUL al II-lea (30 de puncte)

1. Programare: n-ouroboros

Soluție C++:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <sstream>

using namespace std;

int ouroboros(string s) {
    int len = s.length();
    int max_n = 0;
    for (int n = 1; 2 * n <= len; ++n) {
        string prefix = s.substr(0, n);
        string suffix = s.substr(len - n, n);
        if (prefix == suffix) {
            max_n = n;
        }
    }
    return max_n;
}

string to_lower_str(string s) {
    string res = s;
    for (char &c : res) {
        c = tolower(c);
    }
    return res;
}

int main() {
    string text;
    if (!getline(cin, text)) return 0;

    stringstream ss(text);
    string word;
    vector<string> words;
    int max_overall_n = 0;
    vector<string> best_words;

    while (ss >> word) {
        words.push_back(word);
        string lower_w = to_lower_str(word);
        int n = ouroboros(lower_w);
        if (n > max_overall_n) {
            max_overall_n = n;
            best_words.clear();
            best_words.push_back(word);
        } else if (n == max_overall_n && n > 0) {
            best_words.push_back(word);
        }
    }

    if (max_overall_n > 0) {
        cout << max_overall_n << endl;
        for (int i = 0; i < best_words.size(); ++i) {
            cout << best_words[i] << (i == best_words.size() - 1 ? "" : " ");
        }
    }
}
```

```

    }
    cout << endl;
} else {
    cout << "NU" << endl;
}

return 0;
}

```

Soluție Pascal:

```

program OuroborosText;
uses sysutils;

var
    textInput: string;
    words, best_words: array[1..100] of string;
    wordCount, bestWordCount, i, max_overall_n, n: integer;
    currWord: string;

function ouroboros(s: string): integer;
var
    len, max_n, n_val: integer;
    prefix, sufix: string;
begin
    len := length(s);
    max_n := 0;
    for n_val := 1 to len div 2 do
        begin
            prefix := copy(s, 1, n_val);
            sufix := copy(s, len - n_val + 1, n_val);
            if prefix = sufix then
                max_n := n_val;
        end;
    ouroboros := max_n;
end;

begin
    readln(textInput);
    wordCount := 0;
    bestWordCount := 0;
    max_overall_n := 0;

    // Împărțire în cuvinte
    currWord := '';
    for i := 1 to length(textInput) do
        begin
            if textInput[i] <> ' ' then
                currWord := currWord + textInput[i]
            else if currWord <> '' then
                begin
                    wordCount := wordCount + 1;
                    words[wordCount] := currWord;
                    currWord := '';
                end;
        end;
    end;
    if currWord <> '' then

```

```

begin
    wordCount := wordCount + 1;
    words[wordCount] := currWord;
end;

for i := 1 to wordCount do
begin
    n := ouroboros(lowercase(words[i]));
    if n > max_overall_n then
    begin
        max_overall_n := n;
        bestWordCount := 1;
        best_words[1] := words[i];
    end
    else if (n = max_overall_n) and (n > 0) then
    begin
        bestWordCount := bestWordCount + 1;
        best_words[bestWordCount] := words[i];
    end;
end;

if max_overall_n > 0 then
begin
    writeln(max_overall_n);
    for i := 1 to bestWordCount do
    begin
        write(best_words[i]);
        if i < bestWordCount then write(' ');
    end;
    writeln;
end
else
    writeln('NU');
end.

```

2. Algoritm eficient: Sumă subset 2023

- **Algoritmul:** Utilizăm un vector caracteristic boolean `possible` de lungime 2024, inițializat cu false și `possible[0] = true`. Citim numerele din fișier și, pentru fiecare număr x care satisface $1 \leq x \leq 2023$, actualizăm posibilitatea sumelor de la 2023 în jos: `possible[v] = possible[v] || possible[v - x]`. Această abordare rulează în $O(M \times 2023)$ unde M este numărul de numere, adică extrem de rapid. Spațiul este constant ($O(1)$).

Soluție C++:

```

#include <iostream>
#include <fstream>
using namespace std;

bool possible[2024];

int main() {
    ifstream fin("titu2023.in");
    if (!fin) { cout << "NU\n"; return 0; }
    possible[0] = true;
    int x;
    while (fin >> x) {

```

```

        if (x >= 1 && x <= 2023) {
            for (int v = 2023; v >= x; --v) {
                if (possible[v - x]) possible[v] = true;
            }
        }
    }
    fin.close();
    if (possible[2023]) cout << "DA\n";
    else cout << "NU\n";
    return 0;
}

```

Soluție Pascal:

```

program Suma2023;
var
    fin: text;
    possible: array[0..2023] of boolean;
    x, v: integer;
begin
    assign(fin, 'titu2023.in'); reset(fin);
    for v := 0 to 2023 do possible[v] := false;
    possible[0] := true;
    while not eof(fin) do
        begin
            read(fin, x);
            if (x >= 1) and (x <= 2023) then
                begin
                    for v := 2023 downto x do
                        if possible[v - x] then possible[v] := true;
                    end;
                end;
        end;
    close(fin);
    if possible[2023] then writeln('DA')
    else writeln('NU');
end.

```

II. Rezolvare Titularizare Varianta 3 (17 iulie 2024)

SUBIECTUL I (30 de puncte)

1. Structura de date: Arbori

- **Graf neorientat:** Pereche ordonată $G = (V, E)$, unde V este mulțimea nodurilor și E mulțimea muchiilor.
- **Lanț:** Succesiune de noduri interconectate prin muchii distincte.
- **Ciclu:** Un lanț simplu în care nodul de pornire coincide cu cel de sosire.
- **Proprietăți arbore:**
 1. Conex și fără cicluri.
 2. Fără cicluri și are $N - 1$ muchii.
 3. Conex și are $N - 1$ muchii.
 4. Între oricare două noduri există un lanț simplu unic.

5. Dacă se elimină o muchie, devine neconex; dacă se adaugă o muchie, se formează exact un ciclu.

• **Problemă (Parcurgerea în lăţime a unui arbore):**

► **C++:**

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
vector<int> adj[100];
bool viz[100];
void BFS(int start) {
    queue<int> Q;
    Q.push(start);
    viz[start] = true;
    while(!Q.empty()) {
        int u = Q.front(); Q.pop();
        cout << u << " ";
        for(int v : adj[u])
            if(!viz[v]) { viz[v] = true; Q.push(v); }
    }
}
```

► **Pascal:**

```
program BFSArbore;
const NMAX = 100;
var
    n, i, x, y, start, st, dr, u, v: integer;
    a: array[1..NMAX, 1..NMAX] of integer;
    grad, q: array[1..NMAX] of integer;
    viz: array[1..NMAX] of boolean;
begin
    read(n);
    for i := 1 to n - 1 do
        begin
            read(x, y);
            grad[x] := grad[x] + 1; a[x, grad[x]] := y;
            grad[y] := grad[y] + 1; a[y, grad[y]] := x;
        end;
    read(start);
    st := 1; dr := 1; q[1] := start; viz[start] := true;
    while st <= dr do
        begin
            u := q[st]; st := st + 1;
            write(u, ' ');
            for i := 1 to grad[u] do
                begin
                    v := a[u, i];
                    if not viz[v] then
                        begin
                            dr := dr + 1; q[dr] := v; viz[v] := true;
                        end;
                end;
            end;
        end;
    end.
```

2. Ergonomia postului de lucru

• Noțiuni preliminare:

1. **Sistem de calcul:** ansamblu hardware-software care prelucrează automat date.
2. **Dispozitive periferice de intrare:** tastatură, mouse, scanner; permit introducerea datelor/comenzilor.
3. **Dispozitive periferice de ieșire:** monitor, imprimantă, boxe; permit prezentarea rezultatelor.

• Dispozitive periferice cu impact asupra sănătății:

1. Monitorul:

- **Funcție de bază:** afișează informații textuale/grafice.
- **Influență asupra sănătății:** poziționarea greșită, luminozitatea excesivă sau reflexiile pot provoca oboseală oculară, cefalee și postură incorectă.

2. Tastatura și mouse-ul:

- **Funcție de bază:** introducerea datelor și controlul interfeței.
- **Influență asupra sănătății:** utilizarea îndelungată în poziții forțate poate produce dureri la nivelul încheieturilor, antebrațului și umărului.

• Măsuri ergonomice:

1. Monitorul se poziționează la 50-70 cm de ochi, cu partea superioară aproximativ la nivelul privirii.
2. Scaunul se reglează astfel încât spatele să fie sprijinit, iar coatele și genunchii să formeze un unghi apropiat de 90 de grade.
3. Se fac pauze scurte și regulate, cu relaxarea privirii și mișcări pentru mâini/umeri.

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Lanț muntos (Creastă)

Soluție C++:

```
#include <iostream>
#include <cmath>
using namespace std;

int varf(int n, int a[50][50], int k) {
    int r = k - 1;
    int max_val = a[r][0];
    int max_pos = 0;
    for (int j = 1; j < n; ++j) {
        if (a[r][j] > max_val) {
            max_val = a[r][j];
            max_pos = j;
        }
    }
    for (int j = 0; j < n; ++j) {
        if (j != max_pos && a[r][j] == max_val) return 0;
    }
    for (int j = 0; j < max_pos; ++j) {
        if (a[r][j] >= a[r][j+1]) return 0;
    }
    for (int j = max_pos; j < n - 1; ++j) {
        if (a[r][j] <= a[r][j+1]) return 0;
    }
    return max_pos + 1;
}
```



```

}

int main() {
    int n, a[50][50];
    if (!(cin >> n)) return 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> a[i][j];

    bool e_creasta = true;
    int prev_p = varf(n, a, 1);
    if (prev_p == 0) e_creasta = false;

    for (int k = 2; k <= n && e_creasta; ++k) {
        int curr_p = varf(n, a, k);
        if (curr_p == 0 || abs(curr_p - prev_p) > 1) {
            e_creasta = false;
        }
        prev_p = curr_p;
    }

    if (e_creasta) cout << "DA" << endl;
    else cout << "NU" << endl;

    return 0;
}

```

Soluție Pascal:

```

program LantMuntos;
type
    TMatrice = array[1..50, 1..50] of integer;
var
    n, i, j, prev_p, curr_p: integer;
    a: TMatrice;
    e_creasta: boolean;

function varf(n: integer; var a: TMatrice; k: integer): integer;
var
    max_val, max_pos, col: integer;
    is_mountain: boolean;
begin
    max_val := a[k, 1];
    max_pos := 1;
    for col := 2 to n do
        if a[k, col] > max_val then
            begin
                max_val := a[k, col];
                max_pos := col;
            end;

    is_mountain := true;
    for col := 1 to n do
        if (col <> max_pos) and (a[k, col] = max_val) then
            is_mountain := false;

    for col := 1 to max_pos - 1 do

```

```

    if a[k, col] >= a[k, col + 1] then
        is_mountain := false;

    for col := max_pos to n - 1 do
        if a[k, col] <= a[k, col + 1] then
            is_mountain := false;

    if is_mountain then varf := max_pos
    else varf := 0;
end;

begin
    readln(n);
    for i := 1 to n do
        for j := 1 to n do
            read(a[i, j]);

    e_creasta := true;
    prev_p := varf(n, a, 1);
    if prev_p = 0 then e_creasta := false;

    i := 2;
    while (i <= n) and e_creasta do
    begin
        curr_p := varf(n, a, i);
        if (curr_p = 0) or (abs(curr_p - prev_p) > 1) then
            e_creasta := false;
        prev_p := curr_p;
        i := i + 1;
    end;

    if e_creasta then writeln('DA')
    else writeln('NU');
end.

```

2. Algoritm eficient: Tije discuri

- **Algoritmul:** Rămâne o listă a diametrelor din vârful fiecărei tije ocupate. Această listă este întotdeauna sortată crescător. Căutăm binar prima tijă adecvată. Complexitate: $O(N \log N)$ timp și $O(N)$ spațiu.

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu2024.txt");
    if (!fin) return 1;
    vector<int> R;
    int d;
    while (fin >> d) {
        auto it = lower_bound(R.begin(), R.end(), d);
        if (it == R.end()) R.push_back(d);
    }
}

```

```

        else *it = d;
    }
    fin.close();
    cout << R.size() << endl;
    return 0;
}

```

Soluție Pascal:

```

program TijeDiscuri;
var
    fin: text;
    R: array[1..100000] of integer;
    num_rods, d, pos, st, dr, mid: integer;
begin
    assign(fin, 'titu2024.txt'); reset(fin);
    num_rods := 0;
    while not eof(fin) do
        begin
            read(fin, d);
            st := 1; dr := num_rods; pos := -1;
            while st <= dr do
                begin
                    mid := (st + dr) div 2;
                    if R[mid] >= d then
                        begin
                            pos := mid; dr := mid - 1;
                        end
                    else st := mid + 1;
                end;
            if pos = -1 then
                begin
                    num_rods := num_rods + 1;
                    R[num_rods] := d;
                end
            else R[pos] := d;
            end;
        close(fin);
        writeln(num_rods);
    end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)

- **Tip lecție:** Lecție de formare a priceperilor și deprinderilor practice.
- **Caracteristici:**
 1. Accentul cade pe exersarea directă a operațiilor în aplicație, nu doar pe transmiterea teoretică.
 2. Feedbackul este imediat: elevul vede pe ecran efectul schimbării dimensiunii, fontului, stilului bold/italic/subliniat.
- **Mijloc de învățământ:** calculator cu procesor de text (LibreOffice Writer / Microsoft Word) și videoproiector.
- **Metodă didactică:** exercițiul practic dirijat.
- **Formă de organizare:** activitate individuală la calculator, cu momente frontale de demonstrație.

- **Activitate de învățare:** formatarea unui text brut pentru a obține un anunț școlar lizibil.
- **Scenariu didactic:**
 - Profesorul proiectează un text neformatat și demonstrează selectarea textului, schimbarea dimensiunii fontului și aplicarea stilurilor bold, italic și subliniat.
 - Elevii primesc același text și aplică cerințe graduale: titlul cu font mai mare și bold, cuvintele-cheie italic, termenul-limită subliniat.
 - Profesorul verifică lucrările, corectează folosirea excesivă a stilurilor și discută rolul lizibilității.
 - Elevii compară două variante ale documentului și justifică alegerea formatării finale.

2. Evaluare: Itemi obiectivi (Alegere multiplă)

- **Avantaje:** corectare rapidă și obiectivă; acoperirea unui volum mare de conținut într-un timp scurt.
- **Dezavantaj:** evaluează mai greu argumentarea, creativitatea și modul complet de rezolvare a unei probleme.
- **Tipuri de itemi obiectivi:**
 1. **Item cu alegere multiplă:** elevul selectează răspunsul corect din mai multe variante; corectarea este foarte clară.
 2. **Item cu alegere duală:** elevul decide între adevărat/fals sau da/nu; este potrivit pentru verificarea unor noțiuni punctuale.
- **Item pentru Backtracking (Secvența A):** Care este structura spațiului stărilor pentru generarea permutărilor? A. Produs cartezian. | B. Vector de lungime constantă. | C. Arbore de căutare. | D. Graf conex. Răspuns: C.
- **Item pentru formatarea textului (Secvența B):** Afirmația „Comanda Bold modifică dimensiunea caracterelor selectate” este adevărată sau falsă? Răspuns: Fals. Bold modifică grosimea/alinierea caracterelor, nu dimensiunea lor.